



Program Studi Teknik Elektro ITB

Nama Kuliah (Kode) : EL2008-Pemecahan Masalah dengan Pemrograman

Tahun / Semester : 2025-2026 / Genap

Modul : 5 – Foundation of Algorithm

Nama Asisten / NIM :

Nama Praktikan / NIM : Muhammad Aqeel Ghani / 13224071

Tugas Pendahuluan

Soal Konsep

1. Graph and Tree

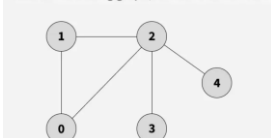
- a. Jelaskan perbedaan Graph dan Tree!

Menurut [1], Graph dan tree adalah dua struktur data fundamental yang digunakan di computer science untuk merepresentasikan hubungan antara objek. Namun, terdapat perbedaan antara mereka. Graph adalah struktur data yang terdiri dari node dan edge (yang mengkoneksikan antara node). Node merepresentasikan entitas, sedangkan edge merepresentasikan hubungan antara entitas. Sedangkan tree adalah struktur data hierarchical yang terdiri dari node dikoneksikan dengan edge. Setiap node dapat memiliki banyak node anak, tetapi hanya memiliki satu node orang tua. Node paling atas suatu tree disebut sebagai root node. Perbedaan pertama antara graph dan tree adalah graph dapat memiliki banyak jalur yang menghubungkan dua node, sedangkan tree hanya memiliki satu jalur yang menghubungkan dua node. Lalu graph tidak memiliki root node, sedangkan tree memiliki root node. Graph juga dapat memiliki banyak edge yang beragam, sedangkan tree jika memiliki n node maka edge nya hanya sebanyak $n-1$.

- b. Jelaskan apa itu depth first search (DFS) dan breadth first search (BFS), lalu tuliskan contoh implementasi sederhana dari keduanya!

Menurut [2], Depth First Search (DFS) dan Breadth First Search (BFS) dua algoritma fundamental yang digunakan untuk menjelajahi dan mencari node graph dan tree. DFS melakukan penjelajahan dengan mulai di root node lalu menjelajahi node sedalam mungkin hingga mencapai node yang tidak memiliki node tetangga yang belum dijelajahi. Sedangkan BFS melakukan penjelajahan dengan memasuki seluruh node di level yang sama terlebih dahulu sebelum lanjut ke level selanjutnya.

Given the following graph, find its DFS traversal.



Gambar Graph DFS

Berikut merupakan contoh DFS dari [3]:

```

// Referensi
// https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/

#include <stdio.h>

#define V 5

void dfsRec(int adj[V][V], int visited[V], int s, int res[V], int *idx) {
    visited[s] = 1;
    res[*idx++] = s;

    // Recursively visit all adjacent vertices
    // that are not visited yet
    for (int i = 0; i < V; i++) {
        if (adj[s][i] && visited[i] == 0)
            dfsRec(adj, visited, i, res, idx);
    }
}

void dfs(int adj[V][V], int res[V]) {
    int visited[V] = {0};
    int idx = 0;
    dfsRec(adj, visited, 0, res, &idx);
}

void addEdge(int adj[V][V], int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1; // undirected
}

int main() {
    int adj[V][V] = {0};

    // creating adjacency list
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 0);
    addEdge(adj, 2, 0);
    addEdge(adj, 2, 3);
    addEdge(adj, 2, 4);

    int res[V];

```

```

// Perform DFS starting from the source vertex 0
dfs(adj, res);

for (int i = 0; i < V; i++)
    printf("%d ", res[i]);

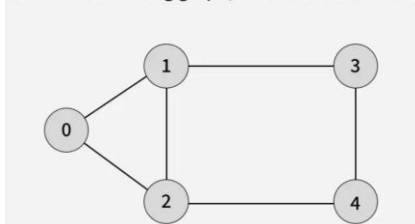
return 0;
}

```

Output:

```
0 1 2 3 4
```

Given the following graph, find its BFS traversal



Gambar Graph BFS

Berikut merupakan contoh BFS dari [4]:

```

// Referensi
// https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/

#include <stdio.h>

#define V 5
#define MAXQ 100

// BFS for single connected component
void bfs(int adj[V][V], int res[V], int *resSize) {
    int visited[V] = {0};
    int q[MAXQ];
    int front = 0, rear = 0;
    int src = 0;
    visited[src] = 1;
    q[rear++] = src;

    while (front < rear) {
        int curr = q[front++];
        res[(*resSize)++] = curr;

        // visit all the unvisited
        // neighbours of current node
        for (int x = 0; x < V; x++) {

```

```

        if (adj[curr][x] && !visited[x]) {
            visited[x] = 1;
            q[rear++] = x;
        }
    }
}

void addEdge(int adj[V][V], int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1; // undirected
}

int main() {
    int adj[V][V] = {0};

    // creating adjacency list
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 0);
    addEdge(adj, 2, 0);
    addEdge(adj, 2, 3);
    addEdge(adj, 2, 4);

    int res[V];
    int resSize = 0;

    bfs(adj, res, &resSize);

    for (int i = 0; i < resSize; i++)
        printf("%d ", res[i]);

    return 0;
}

```

Output:

```
0 1 2 3 4
```

c. Jelaskan DFS pre-order, in-order, dan post-order!

Menurut [5], DFS pre-order, in-order, dan post-order merupakan beberapa teknik tree traversal yang merupakan proses mengakses tiap node tree sekali dengan urutan yang spesifik. Pre-order memiliki urutan root→left→right. Pre-order digunakan untuk membuat copy dari tree

dan memperoleh prefix expression dari expression tree. In-order memiliki urutan left→root→right. In-order digunakan dalam binary search tree untuk memberikan node dalam order non-decreasing dan mengevaluasi ekspresi aritmatika yang tersimpan dalam expression tree. Post-order memiliki urutan left→right→root. Post-order digunakan untuk menghapus tree, mendapatkan postfix expression dari expression tree, dan membantu dalam garbage collection.

2. Strategi Algoritma

- a. Jelaskan apa itu backtracking dan divide and conquer serta berikan contoh permasalahan sederhana yang dapat diselesaikan dengan kedua strategi tersebut!

Menurut [6], backtracking adalah teknik algoritma pemecahan masalah yang mencari solusi secara bertahap dengan mencoba berbagai opsi dan revert/undoing jika mencapai titik ujung. Strategi backtracking adalah Choose→Explore→Check Validity→Repeat. Tujuan strategi ini adalah untuk memastikan tidak membuang waktu mencari solusi melalui jalur yang tidak mungkin. Contoh permasalahan sederhana yang dapat diselesaikan strategi backtracking adalah N Queens Problem yaitu permasalahan untuk meletakkan N queens dalam NxN papan catur sehingga tidak ada dua Queens yang saling menyerang.

Menurut [7], divide and conquer adalah teknik algoritma pemecahan masalah yang membagi permasalahan utama menjadi sub permasalahan dan menyelesaikannya satu-persatu dan menggabungkan kembali untuk menemukan solusi permasalahan utama. Strategi Divide and Conquer adalah Divide→Conquer→Merge. Contoh permasalahan yang dapat menggunakan strategi divide and conquer adalah binary search (higher or lower) yaitu algoritma searching yang membandingkan nilai tengah dengan nilai target dan menentukan apakah nilai target lebih tinggi atau lebih rendah dari nilai tengah (membagi menjadi dua).

- b. Jelaskan minimal 2 buah aplikasi search algorithms dalam permasalahan komputasi kompleks yang riil!

Aplikasi pertama yang akan dijelaskan adalah aplikasi A* di path finding entity Minecraft berdasarkan [8]. Path finding entity/mob di Minecraft menggunakan A*, lebih spesifik lagi A* yang di beratkan parameter heuristic sebesar 1.5 kali (dari persamaan : $\min f(n) = g(n) + h(n)$; menjadi : $\min f(n) = g(n) + 1.5h(n)$) dengan tujuan untuk lebih cepat dalam path finding walau pada akhirnya tidak melewati jalur paling cepat. Lalu dalam Minecraft, tidak semua block memiliki nilai yang sama dalam perhitungan A* ini, ini bertujuan supaya mob/entity ini tidak path find melalui hal yang berbahaya (tetapi sistem ini dapat dieksloit).

Aplikasi kedua yang akan dijelaskan adalah aplikasi algoritma djikstra pada Google Maps. Menurut [9] google maps menggunakan djikstra dalam mencari jalan tercepat antar dua tempat. Kondisi traffic dan jarak jalur adalah parameter yang menentukan berat edges antara dua node.

Soal Pemrograman

```
// Soal Pemograman Modul 5
// Muhammad Aqeel Ghani
// 13224071

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Struktur data untuk Node Tree nya
typedef struct Node{
    char Nama[100];
    struct Node *anak1, *anak2;
} Node;

// Fungsi Pencari LCA (Lowest Common Ancestor)
Node* LCA (Node *root, char *nama1, char *nama2, int *kemunculanNama1, int
*kemunculanNama2){
    // Jika merupakan ujung tree
    if (root == NULL) return NULL;

    if ((strcmp(root->Nama, nama1) == 0)) (*kemunculanNama1) = 1;
    if ((strcmp(root->Nama, nama2) == 0)) (*kemunculanNama2) = 1;

    // Jika root merupakan salah satu keturunan yang dicari
    if ((strcmp(root->Nama, nama1) == 0) || (strcmp(root->Nama, nama2) == 0))
return root;

    Node *anak1LCA = LCA(root->anak1, nama1, nama2, kemunculanNama1,
kemunculanNama2);
    Node *anak2LCA = LCA(root->anak2, nama1, nama2, kemunculanNama1,
kemunculanNama2);

    // Jika kedua anak merupakan leluhur dari keturunan yang dicari, maka
root ini merupakan LCA
    // (Berjalan/terpenuhi jika kedua variabel bukan NULL)
    if (anak1LCA && anak2LCA) return root;

    // Jika Ternyata Salah satu anaknya merupakan keturunan yang dicari,
apakah dia juga leluhur keturunan yang lain
    // Jika iya, maka orangtuanya adalah LCA
    else if (anak1LCA != NULL){
```

```

        if ((strcmp(anak1LCA->Nama, nama1) == 0) || (strcmp(anak1LCA->Nama,
nama2) == 0)){
            if (LCA(anak1LCA->anak1, nama1, nama2, kemunculanNama1,
kemunculanNama2) || LCA(anak1LCA->anak2, nama1, nama2, kemunculanNama1,
kemunculanNama2)){
                return root;
            }
        }
    }
else if (anak2LCA != NULL){
    if ((strcmp(anak2LCA->Nama, nama1) == 0) || (strcmp(anak2LCA->Nama,
nama2) == 0)){
        if (LCA(anak2LCA->anak1, nama1, nama2, kemunculanNama1,
kemunculanNama2) || LCA(anak2LCA->anak2, nama1, nama2, kemunculanNama1,
kemunculanNama2)){
            return root;
        }
    }
}

// Mengembalikan pointer anak yang bukan NULL (Jika ada)
return (anak1LCA != NULL) ? anak1LCA : anak2LCA;
}

// Fungsi Create Node
Node* CreateNode(char *nama){
    Node *newNode = malloc(sizeof(Node));

    strcpy(newNode->Nama, nama);
    newNode->anak1 = NULL;
    newNode->anak2 = NULL;

    return newNode;
}

// Fungsi Pembuat Tree (Sesuai dengan ilustrasi soal)
Node* CreateTree(){
    // Generasi Pertama
    Node* root = CreateNode("Yoru");

    // Generasi Kedua
    root->anak1 = CreateNode("Reyna");
    root->anak2 = CreateNode("Killjoy");
}

```

```

// Generasi Ketiga
root->anak1->anak1 = CreateNode("Omen");
root->anak1->anak2 = CreateNode("Breach");

root->anak2->anak1 = CreateNode("Clove");
root->anak2->anak2 = CreateNode("Brimstone");

// Generasi Keempat
root->anak2->anak1->anak1 = CreateNode("Cypher");
root->anak2->anak1->anak2 = CreateNode("Vyse");

return root;
}

// Fungsi Main
int main(){
    // Pendefinisian dan inisialisasi Variabel
    Node *pohonKeluarga = CreateTree();
    char nama1[100], nama2[100];
    int kemunculanNama1 = 0, kemunculanNama2 = 0;

    // Pengambilan Input
    printf("Masukkan orang pertama: ");
    scanf("%s", nama1);

    printf("Masukkan orang kedua: ");
    scanf("%s", nama2);

    // Mencari LCA
    Node *lca = LCA(pohonKeluarga, nama1, nama2, &kemunculanNama1,
&kemunculanNama2);

    // Print hasil
    if ((strcmp(pohonKeluarga->>Nama, nama1) == 0) || (strcmp(pohonKeluarga-
>Nama, nama2) == 0)) printf("%s merupakan leluhur paling sepuh jadi gak punya
leluhur lagi bos!!!", pohonKeluarga->Nama);
    else if (!(kemunculanNama1 || kemunculanNama2)) printf("%s dan %s siapa
bro?!\n", nama1, nama2);
    else if (!kemunculanNama1) printf("%s siapa bro?!", nama1);
    else if (!kemunculanNama2) printf("%s siapa bro?!", nama2);
    else printf("Leluhr mereka adalah %s.\n", lca->Nama);

```



```
return 0;

}  
  
// Referensi  
// https://www.geeksforgeeks.org/dsa/lowest-common-ancestor-binary-tree-set-1/
```

Input Output :

Masukkan orang pertama: Omen
Masukkan orang kedua: Breach
Leluhur mereka adalah Reyna.

Masukkan orang pertama: Clove
Masukkan orang kedua: Cypher
Leluhur mereka adalah Killjoy.

Masukkan orang pertama: Vyse
Masukkan orang kedua: Asep
Asep siapa bro?!

Masukkan orang pertama: Asep
Masukkan orang kedua: Didin
Asep dan Didin siapa bro?!

Masukkan orang pertama: Yoru
Masukkan orang kedua: Brimstone
Yoru merupakan leluhur paling sepuh jadi gak punya leluhur lagi bos!!!

Referensi

- [1] GeeksforGeeks, "Difference Between Graph and Tree," *GeeksforGeeks*, Jul. 11, 2025. <https://www.geeksforgeeks.org/dsa/difference-between-graph-and-tree/>
- [2] GeeksforGeeks, "Difference between BFS and DFS," *GeeksforGeeks*, Nov. 01, 2025. <https://www.geeksforgeeks.org/dsa/difference-between-bfs-and-dfs/>
- [3] GeeksforGeeks, "Depth First Search or DFS for a Graph," *GeeksforGeeks*, Mar. 28, 2026. <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
- [4] GeeksforGeeks, "Breadth First Search or BFS for a Graph," *GeeksforGeeks*, Jan. 16, 2026. <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
- [5] GeeksforGeeks, "Tree Traversal Techniques," *GeeksforGeeks*, Dec. 06, 2025. <https://www.geeksforgeeks.org/dsa/tree-traversals-inorder-preorder-and-postorder/>
- [6] GeeksforGeeks, "Introduction to Backtracking," *GeeksforGeeks*, Dec. 20, 2025. <https://www.geeksforgeeks.org/dsa/introduction-to-backtracking-2/>

- [7] GeeksforGeeks, "Introduction to Divide and Conquer Algorithm," *GeeksforGeeks*, Jan. 19, 2026. <https://www.geeksforgeeks.org/dsa/introduction-to-divide-and-conquer-algorithm/>
- [8] RedLogic, "Minecraft's Smartest System Is Almost Completely Untapped," *YouTube*, Sep. 11, 2025. <https://www.youtube.com/watch?v=ZgOCxn8AVZA>
- [9] Shrouded Science, "The Science Behind Google Maps | Route Finding Algorithms," *YouTube*, Sep. 16, 2022. <https://www.youtube.com/watch?v=QhcerJZD1Gc> (accessed May 08, 2026).
- [10] GeeksforGeeks, "Lowest Common Ancestor in a Binary Tree," *GeeksforGeeks*, Jan. 21, 2026. <https://www.geeksforgeeks.org/dsa/lowest-common-ancestor-binary-tree-set-1/>

Lampiran

Repositori Github Tugas Pendahuluan Modul 5 Muhammad Aqeel Ghani :

<https://bit.ly/TPModul5PMPAqeel>

